Computergraphik I

Graphikpipeline und Graphikhardware

Prof. Dr. Michael Guthe

Sommersemester 2026

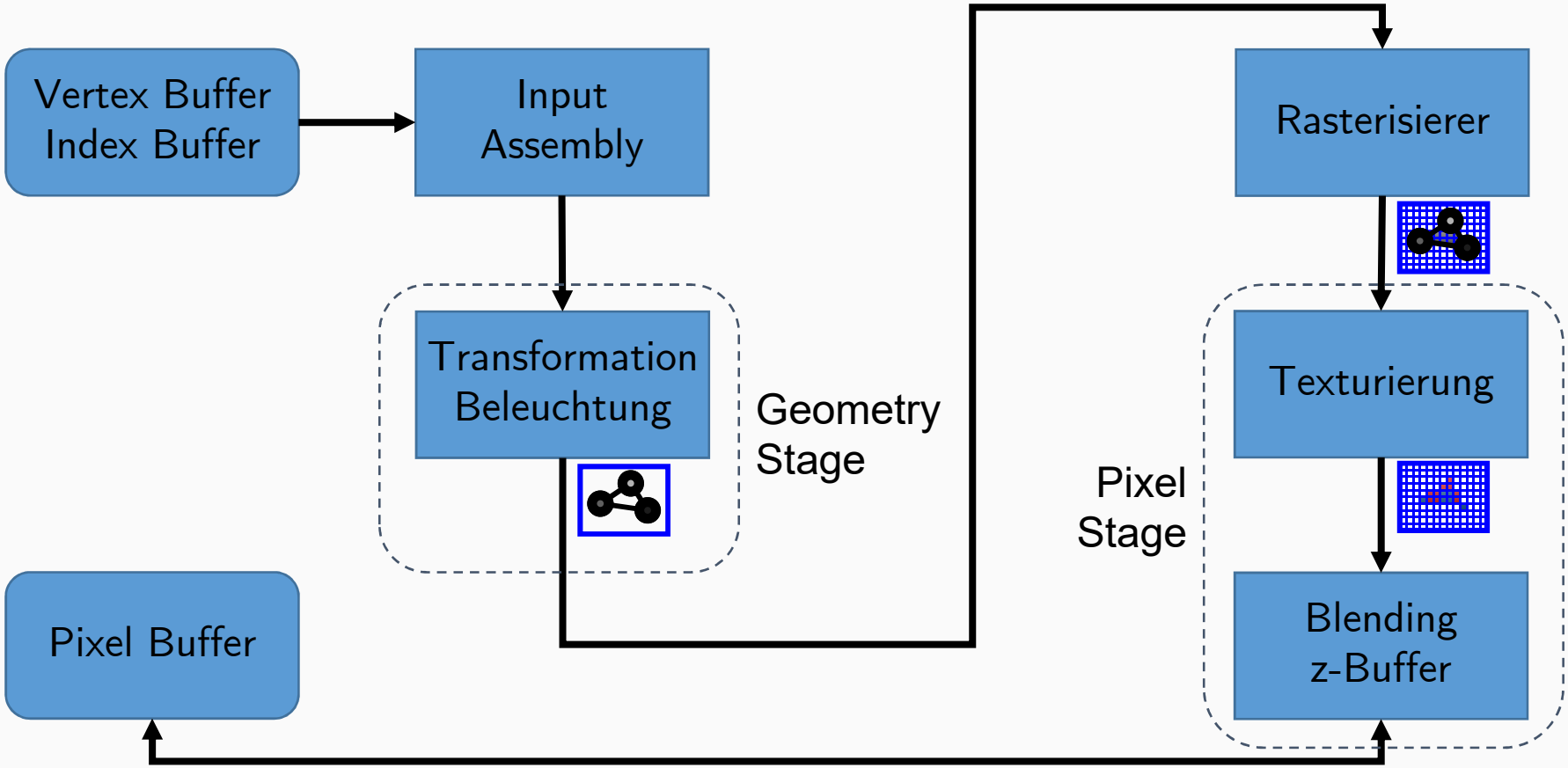
AI 5: Graphische Datenverarbeitung
Universität Bayreuth

Definition: Rendering

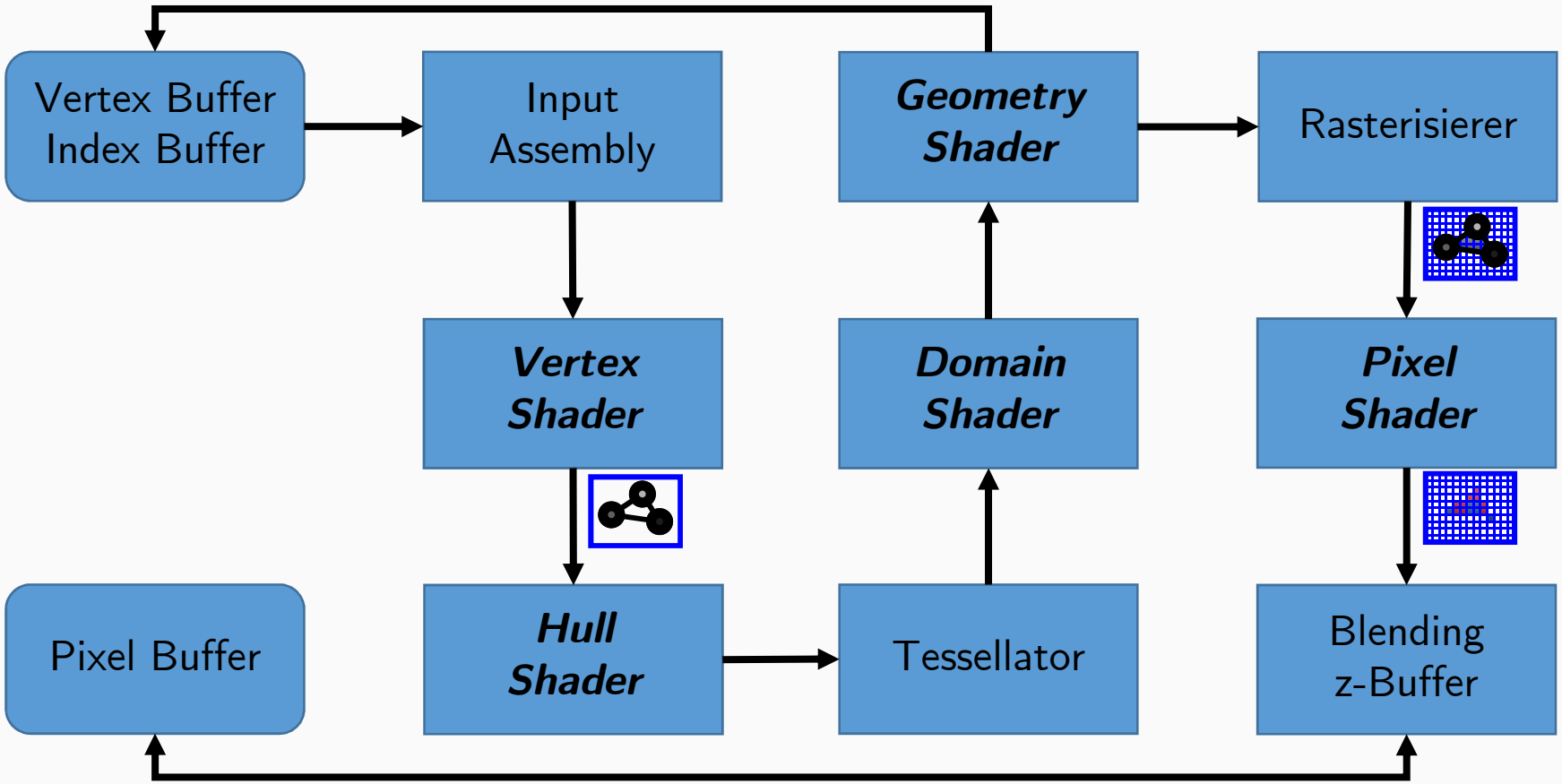
Rendern beziehungsweise 3D Rendern beschreibt in der Computergrafik das Erzeugen von Bildern ausgehend von 2D- oder 3D-Modellen.

Das Echtzeitrendering erfolgt meist mittels einer GPU auf Grundlage der sogenannten Grafikpipeline.

Graphikpipeline (1)



Graphikpipeline (2)



Programmierbare Einheiten (1)

- Reduzierter Befehlssatz gegenüber CPU
- Basisdatentyp ausschließlich Floating Point
  - IEEE 32 Bit Float
  - 16 Bit Float (Half)
  - Integer nur als Arrayindices
- Spezielle Befehle für 4D Vektoren und Matrizen
  - vgl. SIMD-Vektorbefehle aktueller CPUs

Programmierbare Einheiten (2)

- Berechnungen für jeden Vertex, Pixel, etc. gleich
- Entspricht Streamprocessing bei dem Kernel Berechnungen auf Streams durchführen
- Stream (Vertices und Fragmente)
  - Menge von Datenelementen gleichen Typs
- Kernel (Programme)
  - Programmeinheit mit Streams als Parametern
  - Berechnet pro Aufruf aus einem Eingabeelement ein Ausgabeelement
  - Schreibt nicht in globale Daten (parallelisierbar)

Programmierbare Einheiten (3)

sequentieller Algorithmus

```
for(i=0;i<n;i++)
{
  r[i]=func(a[i],b[i],…);
}
```

Kernel

```
r=func(a,b,…);
```

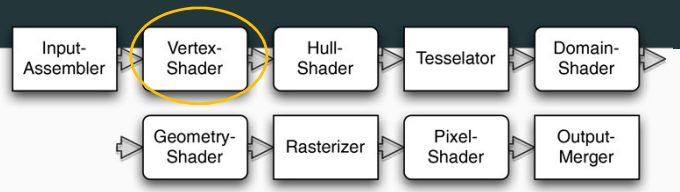
Eingabe-streams

Ausgabe-stream

Programmierbare Einheiten (4)

- Programmiersprachen:
  - Spezielle Assember der Hardwarehersteller
  - Grafik-API-spezifische Hochsprachen (DirectX: HLSL, OpenGL: GLSL)
    - Beide basieren auf C
  - Cg (nVidia) als plattformunabhängige Hochsprache + API Schnittstelle
    - Kann in alle hardware- und API-spezifischen Sprachen übersetzt werden

Vertex Shader



- Eingabe:

  Vertex und Vertexparameter

- Verwendung:

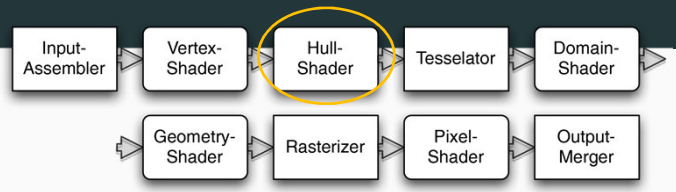
  Transformation, Translation,
  Hinzufügen von Parametern

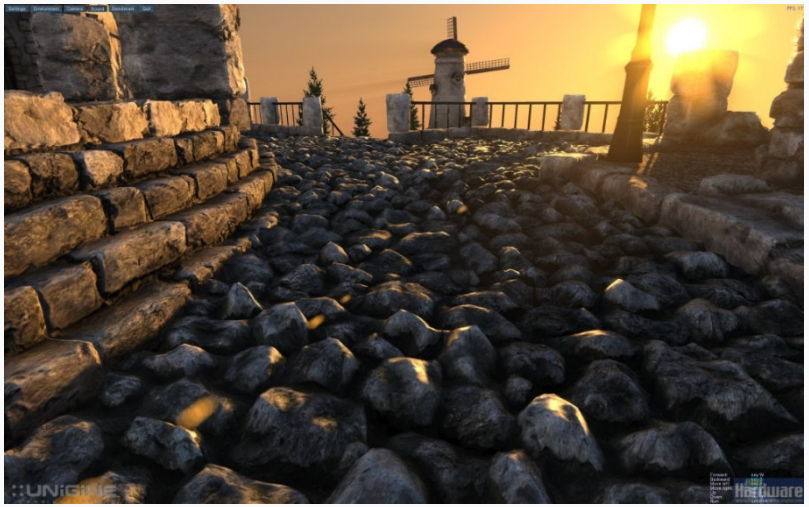
- Ausgabe:

  Vertex und Vertexparameter

Verfügbar ab Shader Model 1.0

Hull Shader





- Eingabe:

  Primitive mit Vertices

  und Parametern

- Verwendung:

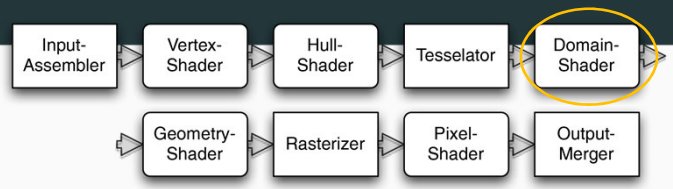
  Berechnen der Parameter
  für den Tessellator, LoD

- Ausgabe:

  Primitive mit Vertices und Parametern
  und Parametern für den Tessellator

Verfügbar ab Shader Model 5.0

Domain Shader



- Eingabe:

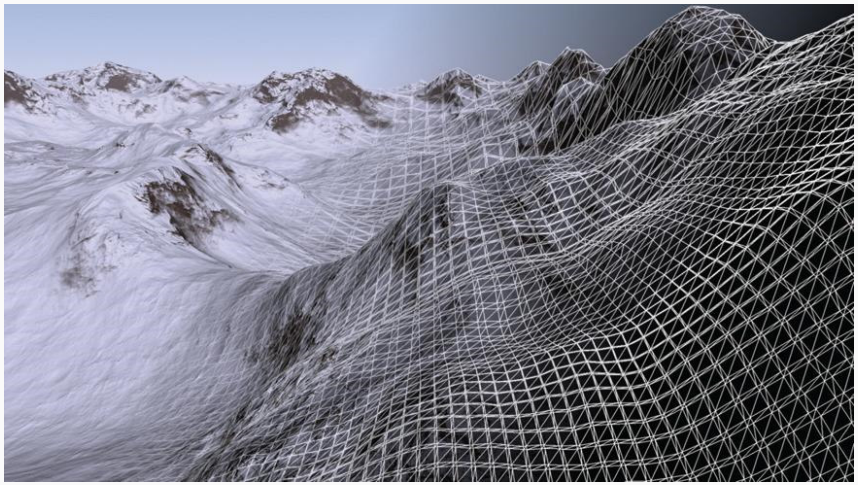
  Vertex aus dem Tessellator mit Informationen über das Primitiv und die Position darin

- Verwendung:

  Verschieben des Vertices

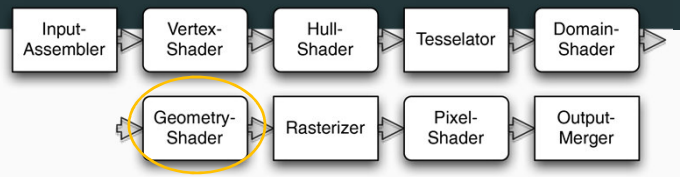
- Ausgabe:

  Vertex mit Parametern



Verfügbar ab Shader Model 5.0

Geometry Shader



- Eingabe:

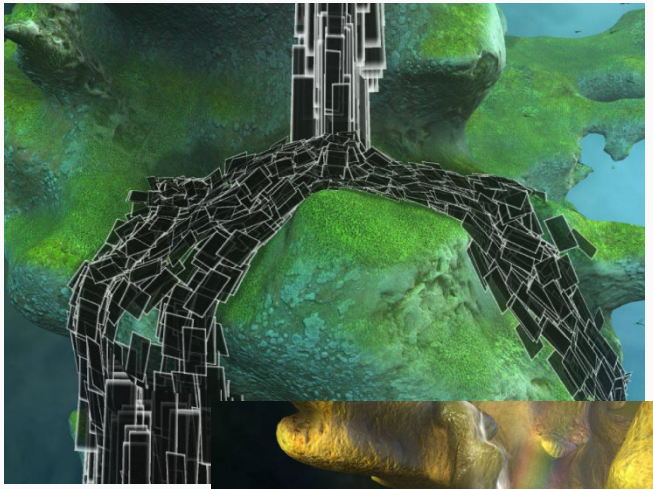
  Primitiv mit zugehörigen Vertices und Parametern

- Verwendung:

  Anpassen, Erzeugen und Aussortieren von Primitiven

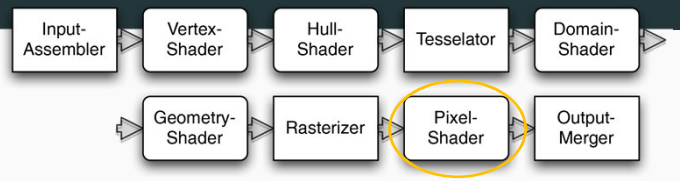
- Ausgabe:

  Primitiv mit zugehörigen Vertices und Parametern, Stream Ausgabe



Verfügbar ab Shader Model 4.0

Pixel Shader



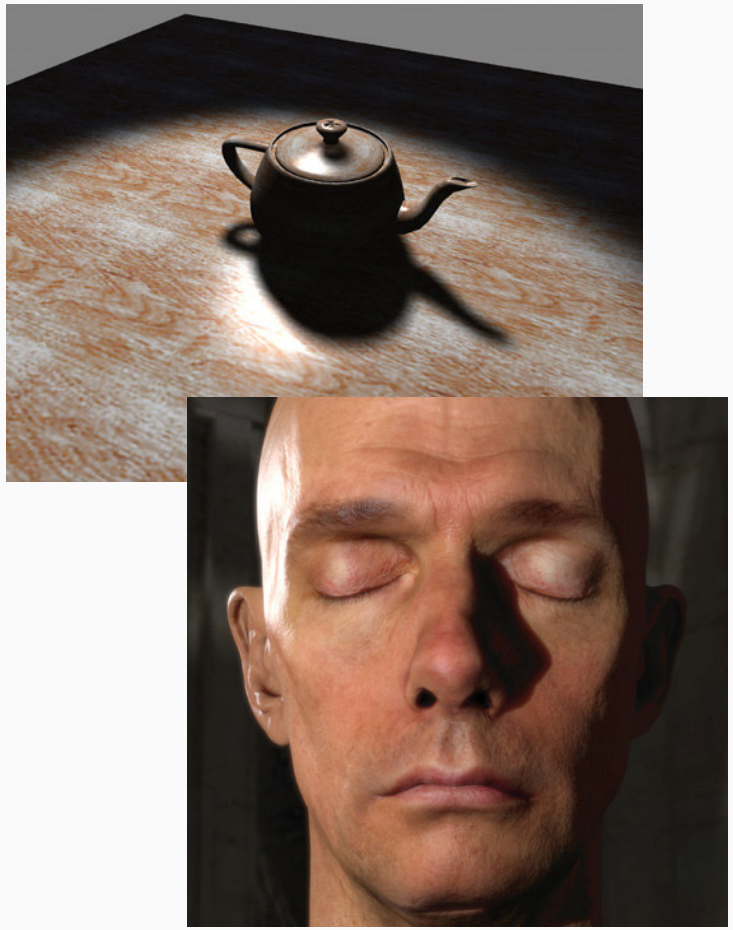
- Eingabe:

  Pixel mit interpolierten Parametern des Primitives

- Verwendung:

  Shading, Operationen im Screenspace, Beleuchtung

- Ausgabe:

  Ein oder mehrere (Farb-) Werte

Verfügbar ab Shader Model 1.1